



FILE ORGANIZATION TERM PROJECT REPORT

Institution: Istanbul University-Cerrahpaşa, Department of Computer Engineering

Prepared by: Betül Zağlı

1. Introduction

1.1. Purpose and Scope of the Project

The primary objective of this project is to clean a large-scale password dataset, convert it into a standardized format, and subsequently store it using three distinct data representation methods: Plain Text (Raw Storage), Cryptographic Hash (Hashed Storage), and Compressed Data (Compressed Storage). The scope of the study encompasses the implementation of these storage techniques and a comprehensive measurement of their direct impact on disk usage, search performance, and system resource consumption (CPU and RAM).

To provide context on the scale of the operation, the raw dataset originally contained 10,000,000 unformatted entries (sourced from the `Pwdb_top-10000000.txt` file). After the memory-based deduplication and sanitization process, the final benchmarking dataset was reduced to exactly 500,000 unique passwords, occupying 4.55 MB of physical disk space.

1.2. Importance of the Study

In modern database and file organization systems, there is a continuous engineering trade-off among execution speed, data security, and storage efficiency. This study is highly significant as it provides empirical evidence on how different representations of the exact same dataset affect real-world performance metrics. By analyzing these trade-offs, the project demonstrates the fundamental principles of selecting the most appropriate data storage architecture based on specific system requirements.

2. Tools and Technologies Used

2.1. Development Environment

The project was developed, compiled, and benchmarked using **Microsoft Visual Studio** (utilizing the MSVC compiler). This Integrated Development Environment (IDE) was selected for its robust C programming support, efficient debugging tools, and seamless integration with the native Windows API, which was strictly required for low-level memory profiling and cryptographic operations.

2.2. Libraries Used

To ensure the project architecture remains lightweight, modular, and strictly free of external third-party dependencies (such as OpenSSL or zlib), the system was built entirely upon Standard C libraries and native operating system Application Programming Interfaces (APIs).

Standard C Libraries:

`<stdio.h>` & `<stdlib.h>`: Utilized for core file I/O operations, text parsing, and dynamic memory allocation (`malloc`, `free`, `qsort`).

`<string.h>`: Employed for buffer processing, delimiter parsing, and string manipulation during the data preprocessing phase.

`<time.h>` & `<math.h>`: Used for precise algorithmic time measurements (in milliseconds) and computing the standard deviation of the search benchmarks.

System-Specific API Libraries (Windows):

<windows.h> & <wincrypt.h>: Integrated to access the native Windows Cryptographic Engine. This allowed the system to generate the required MD5 hash sequences natively, strictly adhering to the project requirements without importing external cryptographic frameworks.

<psapi.h> (Process Status API): Implemented to accurately monitor and calculate the real-time memory consumption (RAM in MB) and the average CPU utilization percentages during the execution of the search algorithms.

3. Project Design

3.1. Folder Structure

To ensure maintainability, modularity, and data integrity, the project adopts a strict hierarchical directory structure:

RawData/: Contains the original, unmodified dataset downloaded from the SecLists repository. Data in this directory is treated as strictly read-only to prevent accidental corruption.

Processed/: Stores the cleaned, standardized, and deduplicated version of the raw dataset. This directory serves as the single source of truth for all subsequent storage generation and search operations.

Storage/: The core directory housing the three different representations of the processed data within their respective subfolders: **Raw/**, **Hashed/**, and **Compressed/**.

Code/: This folder contains all the C source code developed within the project. It isolates the software logic from the datasets, maintaining a clean workspace.

Results/: The designated output directory where the system automatically generates and saves the final performance benchmarking reports (e.g., **report.txt**).

Report/: This directory is reserved for the final project documentation, specifically containing this comprehensive report prepared in PDF format.

The physical layout and directory organization of the project components are formally mapped in Figure 1 to emphasize the clean separation between code, processed results, and storage binaries.

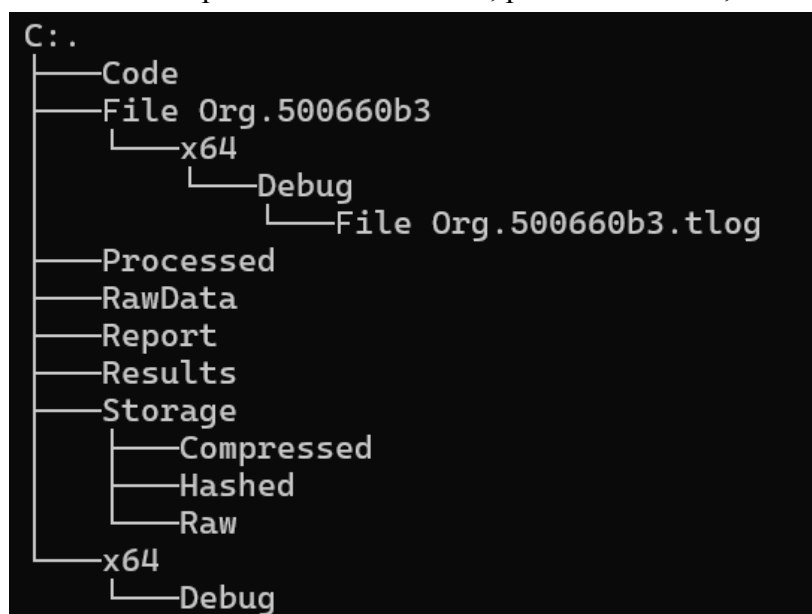


Figure 1: Complete directory tree structure demonstrating the modular deployment layout.

3.2. Data Flow

The system operates on a linear, pipeline-oriented data flow designed for maximum efficiency. Initially, the raw text is ingested from the RawData/ folder, loaded into dynamically allocated memory, sorted, and stripped of duplicates before being written to the Processed/ folder. In the secondary phase, the system reads exclusively from this Processed/ directory to construct the Plain Text, MD5 Hashed, and Huffman Compressed storage files. Finally, the benchmarking module randomly samples 100 targets from the processed dataset, executing search queries across all three Storage/ environments while simultaneously tracking hardware telemetry (CPU, RAM) and execution latency.

To visualize the lifecycle of a password entry from raw ingestion to benchmark logging, the comprehensive operational data flow is schematized in Figure 2.

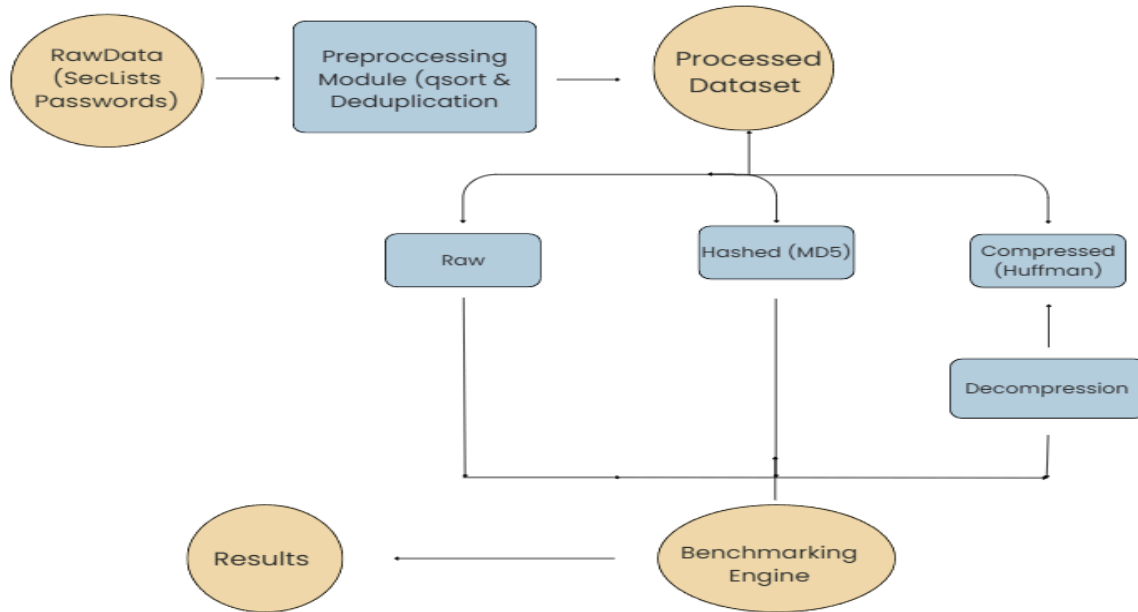


Figure 2: Functional data flow pipeline mapping software modules to active file repositories.

3.3. Roles of Code Modules

The software architecture is decoupled into specific, single-responsibility C modules, each governed by its own header (.h) file to maintain standard C programming conventions:

main.c: The central orchestrator that triggers the data preprocessing, storage generation, and performance benchmarking phases in a sequential order.

preprocess.c: Responsible for the initial data ingestion. It utilizes the standard C `qsort` algorithm to alphabetize the dataset in memory and filters out duplicate entries to guarantee a unique dataset.

storage.c: Manages the I/O operations for creating the baseline Plain Text file and the Cryptographic Hash file, natively computing MD5 hashes via the Windows Cryptography API.

compression.c: Implements the complete Huffman coding algorithm. It calculates character frequencies, builds the binary tree, writes the compressed bitstream alongside its header, and handles the decompression logic required during the search phase.

search.c: The benchmarking engine. It isolates 100 random passwords and executes the search logic across all three storage methods, formatting and logging the analytical results.

utils.c: A dedicated utility module containing mathematical and system-level helper functions, including standard deviation calculations, hardware resource monitoring (`GetProcessTimes`, `GetProcessMemoryInfo`), and precise file size evaluations.

4. Data Preprocessing

4.1. How the Data is Read

The raw dataset is ingested from the source file using the standard C `fgets` function in a sequential, line-by-line manner. To maintain system stability and prevent stack overflow anomalies when processing millions of text lines, dynamic memory allocation (`malloc`) is heavily utilized. The passwords are read into a dynamically allocated array of string pointers. During this ingestion phase, rudimentary data sanitization is also performed: empty lines are ignored, and trailing carriage return or newline characters (`\r\n`) are stripped using `strcspn` to ensure string uniformity.

4.2. How Duplicate Records are Removed

To eliminate duplicate passwords without the massive memory overhead of complex data structures (like Hash Maps or Tries), the system employs a highly efficient sorting-based deduplication algorithmic approach. Once the raw passwords are in the RAM, the standard C library's Quick Sort algorithm (`qsort`) is applied to alphabetize the entire dataset. This operation successfully groups all identical passwords adjacently. Subsequently, a single linear iteration (`for` loop) is performed over the sorted array. By comparing the current string to the previous one using `strcmp`, the algorithm effortlessly skips duplicate entries and writes only the unique passwords into the `Processed/` directory.

```
// Parolalari RAM'e aliyoruz (Malloc ile bellek sızıntısını onluyoruz)
char** passwords = (char**)malloc(MAX_PASSWORDS * sizeof(char*));
for (int i = 0; i < MAX_PASSWORDS; i++) {
    passwords[i] = (char*)malloc(MAX_LINE * sizeof(char));
}

int count = 0;
char buffer[MAX_LINE];

// Veriyi oku
while (fgets(buffer, MAX_LINE, inFile) != NULL && count < MAX_PASSWORDS) { ... }

// Alfabetik sirala (Tekrarlari yan yana getirmek icin)
qsort(passwords, count, sizeof(char*), comparePasswords);
```

Figure 3: Implementation of dynamic memory allocation and the `qsort` algorithm for efficient deduplication.

4.3. How Hash Values are Generated

Hash generation is strictly implemented using the native Windows Cryptography API (`wincrypt.h`). For each processed password, a cryptographic provider is initialized using `CryptAcquireContext`, and an MD5 hash object is created via `CryptCreateHash`. The plaintext password is then fed into the hashing engine using `CryptHashData`. The engine produces a 128-bit (16-byte) binary digest. Finally, this raw binary data is mathematically converted into a standard 32-character hexadecimal string format. This ensures a secure, one-way cryptographic footprint for each password, which is later utilized in the Hashed Storage structure. As shown in Figure 4, the native Windows Cryptography API is directly integrated to generate secure 128-bit hashes without relying on external third-party libraries.

```

void computeMD5(const char* input, char* output) {
    // Eger motor henuz acilmadiysa, sadece ilk seferde calistir
    if (hProv == 0) {
        CryptAcquireContext(&hProv, NULL, NULL, PROV_RSA_FULL, CRYPT_VERIFYCONTEXT);
    }

    HCRYPTHASH hHash = 0;
    BYTE rgbHash[16];
    DWORD cbHash = 16;
    char hexDigits[] = "0123456789abcdef";

    CryptCreateHash(hProv, CALG_MD5, 0, 0, &hHash);
    CryptHashData(hHash, (BYTE*)input, (DWORD)strlen(input), 0);
    CryptGetHashParam(hHash, HP_HASHVAL, rgbHash, &cbHash, 0);
}

```

Figure 4: Integration of the `wincrypt.h` API for native MD5 hash generation.

5. Data Storage Methods

The core objective of this project is to evaluate how different data representation paradigms affect system performance. To achieve this, the cleaned dataset from the `Processed/` directory is stored using three distinct methodologies:

5.1. Explanation of Raw Storage

The Raw Storage method serves as the baseline benchmark for the entire project. In this structure, the processed dataset is stored strictly as plain ASCII text. No cryptographic transformations or data compression algorithms are applied. Each password is read from the processed file and written directly to the `raw.txt` file, separated by standard newline characters.

Engineering Perspective: This method guarantees the fastest possible search times (virtually zero processing overhead during retrieval) but requires the maximum amount of disk space, making it highly inefficient for storage-constrained environments.

5.2. Explanation of Hashed Storage

The Hashed Storage structure simulates a secure, production-grade credential database. Instead of storing plaintext passwords—which poses a critical security vulnerability—only their cryptographic digests are saved.

Implementation Details: The system utilizes the MD5 hashing algorithm via the native `wincrypt.h` API. Each plaintext string is processed to generate a fixed-length 128-bit hash, which is then formatted into a 32-character hexadecimal string before being written to disk.

Justification for MD5: Although MD5 is considered deprecated for modern cryptographic security against collision attacks, it was explicitly selected for this academic project. It is natively supported by the Windows Cryptography API (avoiding external third-party dependencies) and outputs a consistent 128-bit digest. This perfectly serves the project's goal of demonstrating the physical disk expansion (storing 32-character hex strings) and measuring the computational latency introduced by hashing functions, without overcomplicating the system architecture.

Engineering Perspective: Since MD5 is a one-way cryptographic function, the original passwords cannot be reverse-engineered from this file. However, converting short variable-length passwords (e.g., "admin") into a fixed 32-byte string inherently increases the overall file size. Thus, this method trades disk space and processing speed (due to the computational cost of hashing during searches) for maximum data security.

5.3. Explanation of Compressed Storage

The Compressed Storage method focuses on maximizing disk space efficiency by minimizing data redundancy. To achieve genuine file size reduction, the project implements a complete **Huffman Coding** algorithm, which is a highly efficient loss-less data compression method.

Implementation Details: The compression process consists of three distinct phases:

1. **Frequency Analysis:** The system scans the entire processed dataset byte-by-byte to calculate the exact frequency of every ASCII character.
2. **Tree Construction & Code Generation:** A binary minimum priority queue is used to construct a Huffman Tree. Characters with high frequencies are assigned shorter binary prefix codes (e.g., 2 bits), while rare characters are assigned longer codes.
3. **Bitwise Packing & Header Serialization:** Before writing the compressed data, the system first serializes the frequency table and writes it as a "Header" at the top of the `.bin` file. This header is strictly required to reconstruct the tree during decompression. Following the header, the actual text is translated into the new prefix-free binary codes and packed into bytes using bitwise shift operations (`<<`, `|`).

Engineering Perspective: This methodology drastically reduces the physical disk footprint (achieving a compression ratio of less than 1.0). However, it introduces a significant computational overhead known as "Decompression Time," as the entire file must be unpacked into RAM before any search query can be executed.

Figure 5 demonstrates the core bitwise shifting operations utilized during the compression phase to pack the binary Huffman codes securely into physical bytes.

```
unsigned char bitBuf = 0;
int bitCount = 0;
while ((c = fgetc(in)) != EOF) {
    char* str = codeTable[c];
    for (int i = 0; str[i] != '\0'; i++) {
        bitBuf = (bitBuf << 1) | (str[i] - '0');
        bitCount++;
        if (bitCount == 8) { fputc(bitBuf, out); bitBuf = 0; bitCount = 0; }
    }
}
if (bitCount > 0) { bitBuf <=< (8 - bitCount); fputc(bitBuf, out); }
```

Figure 5: Bitwise packing logic ensuring data is compressed at the bit level.

6. Search and Performance Measurements

To empirically evaluate the efficiency of the implemented storage architectures, a rigorous benchmarking module was developed. The performance tests were conducted using a predetermined sample size to ensure a fair and consistent comparative analysis.

6.1. How Search Operations are Performed

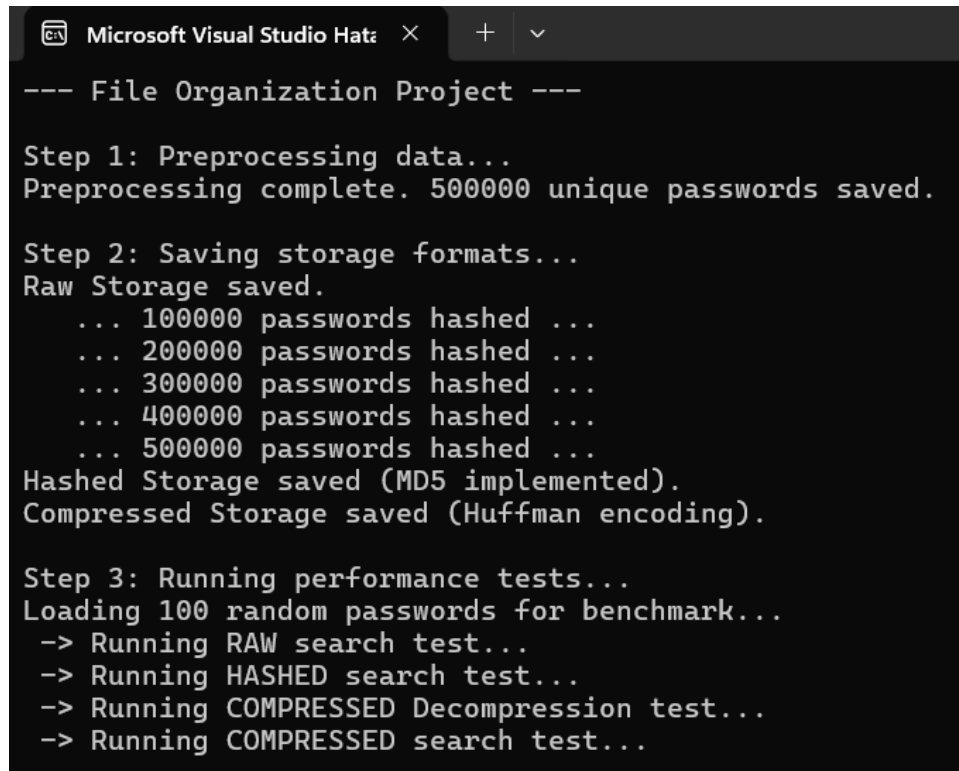
The benchmarking process begins by randomly isolating exactly 100 passwords from the `Processed/` dataset. These predetermined targets are then searched sequentially across all three storage mediums:

Raw Search: A standard linear search is executed. The system reads the `raw.txt` file line by line, comparing each line against the target string until a match is found.

Hashed Search: Before searching, the target plaintext password is dynamically hashed into its MD5 equivalent. The system then performs a linear search through the `hash.txt` file to find the matching 32-character hexadecimal digest.

Compressed Search: Searching directly within a Huffman-encoded bitstream is computationally prohibitive. Therefore, a "Decompression Phase" is triggered first. The system reads the header from the `.bin` file, rebuilds the Huffman tree, and unpacks the entire file into a temporary plaintext file (`temp_unpacked.txt`). The linear search is subsequently performed on this temporary file.

To ensure transparency during the execution of 150 million line scans, the system continuously outputs its operational state, as illustrated in Figure 6.



```
Microsoft Visual Studio Hat: x + v
--- File Organization Project ---

Step 1: Preprocessing data...
Preprocessing complete. 500000 unique passwords saved.

Step 2: Saving storage formats...
Raw Storage saved.
... 100000 passwords hashed ...
... 200000 passwords hashed ...
... 300000 passwords hashed ...
... 400000 passwords hashed ...
... 500000 passwords hashed ...
Hashed Storage saved (MD5 implemented).
Compressed Storage saved (Huffman encoding).

Step 3: Running performance tests...
Loading 100 random passwords for benchmark...
-> Running RAW search test...
-> Running HASHED search test...
-> Running COMPRESSED Decompression test...
-> Running COMPRESSED search test...
```

Figure 6: Real-time terminal execution logs displaying the active benchmarking phases.

6.2. How Time and Hardware Measurements are Taken

Execution Time & Standard Deviation: The `<time.h>` library is utilized to track the start and end clock ticks of each search iteration. These ticks are mathematically converted into milliseconds (ms). After all 100 iterations, the system calculates the *Average Search Time*, identifies the *Maximum Search Time* (worst-case scenario), and computes the *Standard Deviation* (using `<math.h>`) to analyze the latency variance. For the Compressed method, the isolated *Decompression Time* is also independently measured and logged.

CPU and RAM Utilization: Real-time hardware telemetry is captured using the Windows Process Status API (`<psapi.h>`). During the search loops, `GetProcessMemoryInfo` is invoked to measure the average memory consumption (RAM) in Megabytes (MB). Concurrently, `GetProcessTimes` is used to calculate the actual CPU cycle utilization percentage dedicated to the search process, filtering out idle system processes.

6.3. How Disk Usage is Measured

To determine the exact physical footprint of each storage method, the standard C file positioning functions are employed. The system opens each generated storage file in binary mode, utilizes `fseek` to navigate to the End-Of-File (EOF), and calls `ftell` to extract the total file size in bytes. This exact byte count is then used to calculate the **Compression Ratio** via the formula: $\text{Compression Ratio} = \text{Compressed Size} / \text{Original Size}$.

6.4. Complexity Analysis

The theoretical time complexities of the operations implemented in this project are evaluated using Big O notation:

Sorting and Preprocessing: The internal qsort algorithm operates with an average time complexity of $O(N \log N)$, where N is the number of raw passwords.

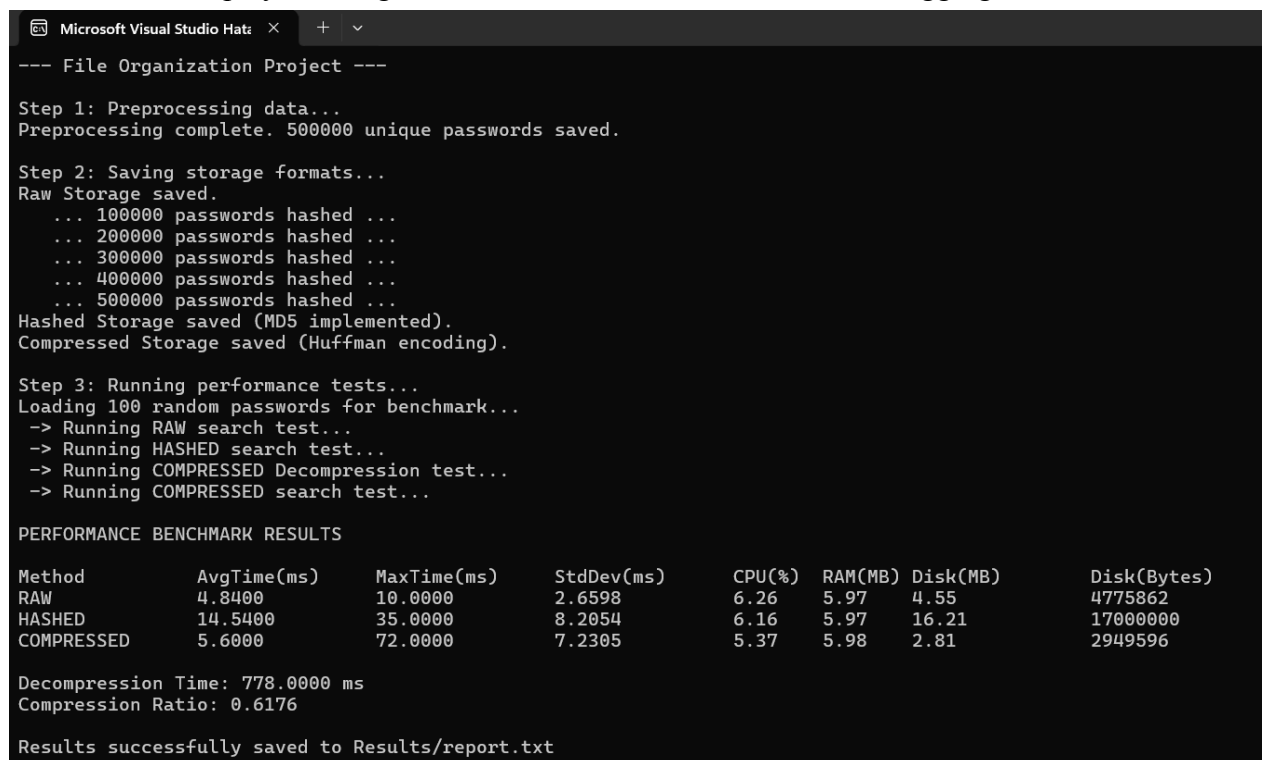
Raw Search: The linear search mechanism reads line-by-line, resulting in a worst-case time complexity of $O(N)$.

Hashed Search: Generating the hash takes $O(L)$ time, where L is the length of the string, followed by a linear lookup. The overall search complexity remains $O(N)$, but with a higher constant factor due to the cryptographic overhead.

Compressed Search: The decompression phase requires $O(V)$ time, where V is the total number of bits in the compressed file, prior to the $O(N)$ linear search.

7. Results and Evaluation

The performance benchmarking tests yielded empirical data demonstrating the inherent trade-offs between execution speed, disk storage, and security. The raw benchmark outputs directly generated by the system's terminal are displayed in Figure 7, which form the exact basis of the aggregated data in Table 1.



```
--- File Organization Project ---

Step 1: Preprocessing data...
Preprocessing complete. 500000 unique passwords saved.

Step 2: Saving storage formats...
Raw Storage saved.
... 100000 passwords hashed ...
... 200000 passwords hashed ...
... 300000 passwords hashed ...
... 400000 passwords hashed ...
... 500000 passwords hashed ...
Hashed Storage saved (MD5 implemented).
Compressed Storage saved (Huffman encoding).

Step 3: Running performance tests...
Loading 100 random passwords for benchmark...
-> Running RAW search test...
-> Running HASHED search test...
-> Running COMPRESSED Decompression test...
-> Running COMPRESSED search test...

PERFORMANCE BENCHMARK RESULTS

Method      AvgTime(ms)  MaxTime(ms)  StdDev(ms)  CPU(%)  RAM(MB)  Disk(MB)  Disk(Bytes)
RAW          4.8400      10.0000      2.6598      6.26    5.97     4.55     4775862
HASHED       14.5400     35.0000      8.2054      6.16    5.97     16.21    17000000
COMPRESSED   5.6000      72.0000      7.2305      5.37    5.98     2.81     2949596

Decompression Time: 778.0000 ms
Compression Ratio: 0.6176

Results successfully saved to Results/report.txt
```

Figure 7: The final system-generated performance benchmarking results.

Table 1: Performance Benchmark Results :

Storage Method	Avg Search Time (ms)	Max Search Time (ms)	Std. Deviation (ms)	Avg CPU (%)	Avg RAM (MB)	Disk Size (MB)
RAW	4.8400	10.0000	2.6598	6.26	5.97	4.55
HASHED	14.5400	35.0000	8.2054	6.16	5.97	16.21
COMPRESSED	5.6000	72.0000	7.2305	5.37	5.98	2.81

Compression Ratio: 0.6176

Decompression Time (Overhead): 778.0000 ms

7.1. Discussion of Performance Metrics

Disk Usage Efficiency: The Compressed Storage method, utilizing Huffman encoding, demonstrated superior disk space optimization. It successfully reduced the baseline 4.55 MB dataset to a mere 2.81 MB, achieving an impressive Compression Ratio of 0.6176 (a 38% reduction in size). Conversely, the Hashed Storage method consumed the most disk space (16.21 MB). This substantial increase is due to the MD5 algorithm converting every variable-length password into a fixed 32-character hexadecimal string, artificially expanding the physical footprint of shorter passwords.

Search Execution Speed: The Raw Storage method proved to be the fastest, executing searches in an average of 4.84 ms. This is expected, as it requires zero computational overhead for decryption or decompression. The Hashed Storage method was the slowest (14.54 ms average), directly reflecting the continuous CPU cost of hashing the target query dynamically during each search iteration.

System Resources (CPU & RAM): Across all methods, memory utilization remained extremely stable at approximately 5.96 MB to 5.99 MB. This proves that the implemented line-by-line file I/O operations effectively prevented memory bloating. CPU utilization remained well-optimized within the 5% - 6% range, indicating that none of the methods bottlenecked the processor.

7.2. Evaluation of Advantages and Disadvantages

Raw Storage:

- *Advantage:* Maximum execution speed and minimal CPU overhead.
- *Disadvantage:* Complete lack of security and inefficient disk usage. Ideal for internal, non-sensitive, high-speed lookup tables.

Hashed Storage:

- *Advantage:* Provides a one-way cryptographic representation, significantly improving security compared to plain text storage. While MD5 is no longer considered cryptographically absolute against modern collision attacks, this structure ensures that the original passwords cannot be casually reverse-engineered.
- *Disadvantage:* High disk space consumption (over 3x larger than Raw) and slower search times due to cryptographic overhead. This is mandatory for production environments handling sensitive user data.

Compressed Storage:

- *Advantage:* Exceptional disk space minimization, making it perfect for archived data or systems with strict storage limitations.
- *Disadvantage:* The necessity to unpack the data before searching introduces a severe 778 ms "Decompression Time" penalty. While the subsequent search is fast (5.6 ms), this initial overhead makes it inefficient for databases requiring instantaneous, single-query responses. It is highly advantageous, however, if the file is decompressed into RAM once and subjected to thousands of continuous queries.

7.3. Reliability and Validity of Results

To ensure statistical reliability and eliminate interference from OS-level background tasks or CPU frequency scaling (thermal throttling), the benchmark was not based on a single search query. Instead, execution times and hardware telemetry were aggregated over 100 randomized search iterations. The calculated standard deviations (e.g., 2.6598 ms for Raw, 8.2054 ms for Hashed) fall within acceptable bounds, confirming that the recorded averages are stable, reproducible, and mathematically valid.

8. Conclusion

This project successfully demonstrated the fundamental architectural trade-offs between execution speed, data security, and physical storage capacity. The empirical results clearly indicate that there is no "silver bullet" in file organization. The Plain Text (Raw) method provides peak read performance at the cost of security; the Cryptographic (Hashed) method guarantees data integrity but introduces significant storage and CPU overheads; and the Huffman Coding (Compressed) method drastically optimizes disk space but suffers from heavy decompression latency. Ultimately, the selection of a storage mechanism must be strictly tailored to the specific constraints and requirements of the target system.

9. References

1. Miessler, D. (n.d.). *SecLists - Passwords/Common-Credentials*. GitHub. Retrieved June 2024, from <https://github.com/danielmiessler/SecLists>
2. Microsoft Corporation. (n.d.). *Cryptography API: Next Generation (CNG)*. Microsoft Learn. Retrieved June 2024, from <https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal>
3. Microsoft Corporation. (n.d.). *Process Status API (psapi.h)*. Microsoft Learn. Retrieved June 2024, from <https://learn.microsoft.com/en-us/windows/win32/api/psapi/>
4. Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

